

Analisi del Report – Modulo 3 – OpenMP

Walkthrough completo del report, sezione per sezione

Partitioned Hash Join with Duplicates

SPM – Università di Pisa – Prof. M. Torquati – A.A. 2025-2026

Walkthrough completo del report, sezione per sezione dall’inizio alla fine, con la motivazione di ogni scelta, la derivazione di ogni numero, la lettura di ogni grafico/tabella, il confronto con baseline/modulo precedente, i *risultati misurati* dove eseguibili senza cluster, e una sezione finale di ulteriori esperimenti. La struttura segue l’ordine del report originale, così puoi presentarlo scorrendo il PDF mentre mostri il documento al prof.

Contents

Come usare questo documento	2
1 Introduction (sezione 1 del report)	2
2 Algorithm and OpenMP Strategies (sezione 2)	2
2.1 Three-phase Structure	2
2.2 Loop-level Variant	2
2.3 Task Variant with LPT	2
2.4 Targeted Optimisations	2
3 Skewed Workload (sezione 3)	3
4 Validation (sezione 4)	3
5 Experimental Evaluation (sezione 5)	3
5.1 Strong Scaling	3
5.2 Weak Scaling e Phase Breakdown	4
5.3 Schedule Sensitivity	4
5.4 Comparison with Module 2	4
6 Discussion (sezione 6)	4
7 Ulteriori esperimenti da fare	4

Come usare questo documento

Segue l'ordine del report (Introduction → Algorithm and OpenMP Strategies → Skewed Workload → Validation → Experimental Evaluation → Discussion). Per ogni sezione: cosa dice, perché, da dove vengono i numeri, come difenderlo.

1 Introduction (sezione 1 del report)

Il report dichiara: stesso algoritmo di M2, riscritto in OpenMP, con due varianti (loop, task) che parallelizzano *ogni* fase con un costrutto diverso; l'obiettivo è valutare quale costrutto conviene sotto carico bilanciato e skewed, e confrontarsi con i `std::thread` di M2.

Motivazione della scelta

Perché tenere il costrutto coerente in tutte le fasi. Per isolare il costo del costrutto: mescolando `for` e `task` non sapresti attribuire le differenze. Loop-ovunque vs task-ovunque misura il modello. Tesi del modulo: loop vince su regolare, task su skew.

2 Algorithm and OpenMP Strategies (sezione 2)

2.1 Three-phase Structure

Il report richiama le tre fasi (Histogram, Scatter, Join), la stessa hash di Fibonacci e la FlatCountMap (capacità $\geq 2|R_p|$, load factor $< 50\%$).

2.2 Loop-level Variant

```
#pragma omp parallel for schedule(dynamic,1) reduction(+:join_count,checksum1,
checksum2)
for(uint32_t pid=0; pid<P; ++pid){ JoinResult r=join_one_partition(Rpart,Spert,pid);
... }
```

Motivazione della scelta

Una iterazione = una partizione. `dynamic,1` consegna una partizione alla volta a domanda perché il costo per partizione è ignoto e variabile. La `reduction` dà somme parziali private (no contesa, no lock). La macro `RUNTIME_SCHEDULE` la trasforma in `schedule(runtime)` (legge `OMP_SCHEDULE`), binario dello `schedule-sweep`.

2.3 Task Variant with LPT

Da dove viene il numero (derivazione)

LPT col bound. Ordina le partizioni per $|R_p| + |S_p|$ decrescente, sottomette dalle più pesanti (`single nowait`). È greedy per il makespan (Graham 1969): $\leq (\frac{4}{3} - \frac{1}{3m})OPT$. I task grossi in coda per primi non finiscono soli sul percorso critico. `nowait`: gli altri thread consumano i task mentre il master li crea. Task *tied*: l'indice di thread resta stabile.

2.4 Targeted Optimisations

Il report elenca tre ottimizzazioni: prefetch, split intra-partizione, NUMA first-touch.

Da dove viene il numero (derivazione)

Split intra-partizione e il tetto H/T . Con H partizioni calde indivisibili, al massimo H thread lavorano $\Rightarrow$ efficienza $\leq H/T$ ($4/16 = 25\%$). Limite strutturale: LPT riordina ma non spezza. “Calda” se $|R_p| + |S_p| > 8\bar{w}$; il *probe* (read-only) è spezzato in $K = \min(T, \lfloor w/\bar{w} \rfloor)$ sub-task (clamp 2) dentro un **taskgroup** (tiene viva la tabella); il *build* resta sequenziale (update su FlatCountMap = race). Così fino a T thread cooperano su una calda, scavalcando H/T .

Motivazione della scelta

Prefetch (12 scatter, 8 probe). Stream ad accesso casuale (destinazione/slot scelti dall’hash), non previsti dal prefetcher HW. Distanza $\approx$ latenza/lavoro-per-iter: scatter leggero $\Rightarrow$ 12 (copre L1 $\rightarrow$ L2); probe pesante $\Rightarrow$ 8 (L2 $\rightarrow$ L3). **NUMA first-touch:** allocatore che fa **resize** senza toccare le pagine; un **parallel for static** di zero-init le tocca per prima con lo stesso partizionamento dello scatter $\Rightarrow$ pagina sul nodo del thread che vi scriverà. **Padding alignas(64) + static_assert:** niente false sharing.

3 Skewed Workload (sezione 3)

Da dove viene il numero (derivazione)

Modello e $f_{\text{hot}} = 0.226$. Mistura di Bernoulli: prob. $\rho = 0.9$ chiave calda (1 di $H = 4$, su partizioni distinte), prob. $1 - \rho$ uniforme. $f_{\text{hot}} = \frac{\rho}{H} + \frac{1-\rho}{P} = \frac{0.9}{4} + \frac{0.1}{128} \approx 0.226$; $f_{\text{cold}} \approx 0.00078$; rapporto $\approx 290\times$. Generatore **splitmix64** con seed da CLI (riproducibile bit a bit). Determinato da (ρ, H, P) , non una manopola.

4 Validation (sezione 4)

Il report: per input piccoli (≤ 500) join naive $O(N^2)$; per input grandi, sweep $\text{modi} \times \text{workload} \times \text{thread} \in \{1, 2, 4, 8, 16\}$, confronto della tripla contro un riferimento (seq per uniforme, par a $T = 1$ per skew, perché il seq non genera skew); inoltre loop e task a $T = 4$ devono dare triple identiche. Checksum ordine-invarianti. Tutto passa sul cluster.

5 Experimental Evaluation (sezione 5)

Setup: nodo *spmcluster* (E5-2640 v2, 32 logici, 2 NUMA), SLURM **--exclusive --cpus-per-task=32**, GCC **-O3 -march=native**, OMP_PROC_BIND=close OMP_PLACES=cores, $N_R = 10^7$, $N_S = 2 \cdot 10^7$, $P = 128$, 3 ripetizioni.

5.1 Strong Scaling

T	loop unif.	task unif.	loop skew	task skew
4	0.164	0.178	0.168	0.147
8	0.112	0.127	0.110	0.094
16	0.070	0.092	0.097	0.087
32	0.086	0.088	0.114	0.089

Table 1: Tempi [s]. $T_{\text{seq}} = 0.802$ s.

Come difenderlo all'orale

Uniforme: loop batte task fino al 31% (T=16). Niente da bilanciare $\Rightarrow$ il task paga creazione/dispatch su histogram/scatter senza beneficio; divario $\sim 8\%$ (T4) $\rightarrow \sim 31\%$ (T16). Loop picca a $11.46\times$ (T=16), degrada a T=20 (team a cavallo fisici/SMT), recupera a T=32. **Skew: si inverte.** T=4 task 0.147 vs loop 0.168 ($\sim 14\%$ da LPT); culmina a T=8 ($\sim 17\%$, entra lo split); $\sim 28\%$ a T=32 (il loop stalla sulle 4 partizioni). Sul join a T=16: 60.9 vs 72.5 ms.

5.2 Weak Scaling e Phase Breakdown

Weak (NR = $2M \times T$): uniforme regge entro un socket (loop $\sim 80\%$ a T=8), poi cala (banda condivisa). Breakdown: uniforme il join domina ($\sim 67\%$ loop a T=8), il task lo comprime al 44–58% (paga i task su histogram/scatter); skew l'imbalance è sul join ($\sim 76\%$ loop, $\sim 70\%$ task a T=16).

5.3 Schedule Sensitivity

Come difenderlo all'orale

A T=8: uniforme le 5 policy entro $\sim 1\%$ (con $P/T = 16$ il carico è identico). Skew: static/static,1/guided,1 ~ 130 ms sul join, le due dynamic ~ 80 ms (-38%): static assegna le 4 calde a chi possiede il range $\rightarrow$ critical path; dynamic le riassegna. Giustifica **dynamic,1** default: non-dominata.

5.4 Comparison with Module 2

Da dove viene il numero (derivazione)

Da dove vengono $11.46\times$ (M3 loop) e $7.86\times$ (M2) a T=16. Baseline seq condiviso $T_{\text{seq}} = 0.802$ s. M3 loop 0.070 s $\Rightarrow 11.46$; M2 0.102 s $\Rightarrow 7.86$. Divario da due fasi: scatter (~ 29 vs ~ 85 ms a T=8, dal prefetch) e histogram (~ 9 vs ~ 35 ms: la completion della `std::barrier` di M2 serializza il merge, l'omp `single` di M3 è parzialmente nascosto). A T=32 convergono (9.33, 9.11, 9.01 $\times$): banda dominante.

Come difenderlo all'orale

Efficienza > 1 per $T \leq 4$: non super-linear. Le ottimizzazioni (prefetch, first-touch) sono nel parallelo ma non nel seq: a T=1 il parallelo è $\sim 1.44\times$ il seq, offset che decade con T. Dichiarato.

6 Discussion (sezione 6)

Il report sintetizza: uniforme il loop (no overhead task), skew il task (LPT+split); schedule **dynamic,1** default; entrambe le varianti OpenMP battono M2 contro il baseline condiviso; a T=32 le tre convergono (banda).

7 Ulteriori esperimenti da fare

Esperimento di supporto1. **LPT vs ordine casuale**

Come: sottometti i task in ordine random invece che LPT, su skew

Misura: makespan/tempo join

Atteso: LPT più veloce

Dimostra: che il guadagno viene da LPT, isolato dallo split.

2. Prefetch ON/OFF

Come: commenta le `__builtin_prefetch` e rimisura

Misura: tempo scatter/probe

Atteso: il salto 85→29 ms sparisce

Dimostra: che il prefetch e' la causa, non altro.

3. Split ON/OFF (K=1)

Come: forza K=1 e rilancia il task su skew

Misura: efficienza skewed

Atteso: crolla verso $H/T \approx 25\%$

Dimostra: empiricamente che lo split rompe H/T .

4. Sweep distanza di prefetch

Come: prova {4, 8, 12, 16, 24}

Misura: tempo di fase

Atteso: ottimo attorno a 12/8

Dimostra: che 12/8 sono tarati sulla latenza, non magici.

5. First-touch ON/OFF

Come: disabilita il zero-init parallelo

Misura: speedup oltre 16 core

Atteso: riappare un dip NUMA

Dimostra: che il first-touch da' la localita' NUMA.

6. Soglia "hot" (4/8/16 $\bar{w}$)

Come: varia la soglia di classificazione calda

Misura: tempo skewed e n. di split

Atteso: compromesso: bassa troppi split, alta ne perdi

Dimostra: che $8\bar{w}$ e' un trade-off motivato.

Collegamento alla teoria

OpenMP loop – schedule, reduction, fork-join (L19); task – task, taskgroup, single nowait, tied/untied (L22); workload balancing e LPT/Graham (L14); affinity e first-touch (L18); prefetching, false sharing.